BİLGİSAYAR BİLİMİNE GİRİŞ · SINAV HAZIRLIK NOTU

Veri Depolama, Veri İşleme ve İşletim Sistemleri

Brookshear, *Computer Science: An Overview* (11. baskı) ders slaytlarının Türkçe, sınav odaklı özeti. Kavramlar, şekiller, çözümlü örnekler ve son tekrar listesi.

Bölüm 1

Veri Depolama

Bit ve kapılar · flip-flop · bellek · disk · ASCII · ikilik sistem · ikiye tümleyen · sıkıştırma

Bölüm 2

Veri İşleme

CPU mimarisi · makine dili · makine döngüsü · mantık işlemleri · G/Ç · paralel mimariler

Bölüm 3

İşletim Sistemleri

Tarihçe · mimari · önyükleme · süreç yönetimi · semafor ve kilitlenme · güvenlik

Kaynak: PPT_Chap_1101 (Chapter 1: Data Storage), PPT_Chap_1102 (Chapter 2: Data Manipulation) ve PPT_Chap_1103 (Chapter 3: Operating Systems) · Terimlerin İngilizce karşılıkları sınavda tanıyabilmeniz için parantez içinde verilmiştir.

İçindekiler

Bölüm 1 — Veri Depolama 3

1.1 Bitler ve Saklanmaları (Boole işlemleri, kapılar, flip-flop, hex) 3

1.2 Ana Bellek · 1.3 Yığın Bellek (disk, optik, flash) 4

1.4 Bilginin Gösterimi (metin, sayı, resim, ses) 6

1.5 İkilik Sistem · 1.6 Tam Sayıları Saklama (ikiye tümleyen, fazlalık) 7

1.7 Kesirler · 1.8 Veri Sıkıştırma · 1.9 İletişim Hataları 9

Bölüm 1 çözümlü sorular 10

Bölüm 2 — Veri İşleme 13

2.1 Bilgisayar Mimarisi (CPU, yazmaçlar, veri yolu, önbellek) 13

2.2 Makine Dili (RISC/CISC, komut türleri, op-code tablosu) 14

2.3 Program Yürütme (makine döngüsü, adım adım örnek) 16

2.4 Aritmetik/Mantık Komutları (maskeleme, döndürme/kaydırma) 17

2.5 Diğer Cihazlarla İletişim · 2.6 Diğer Mimariler 19

Bölüm 2 çözümlü sorular 20

Bölüm 3 — İşletim Sistemleri 22

3.1 Tarihçe · 3.2 İşletim Sistemi Mimarisi (çekirdek, önyükleme) 22

3.3 Faaliyetleri Koordine Etme (süreç, zamanlayıcı, kesme) 24

3.4 Süreçler Arası Rekabet · 3.5 Güvenlik 25

Bölüm 3 çözümlü sorular 28

Son Tekrar: sık karıştırılanlar · terim sözlüğü 30

Bu notu nasıl kullanmalısınız?

Tanım kutuları: ezberlemeniz gereken temel tanımlar.

Sınav kutuları: slaytlarda öne çıkan, soru olarak gelmeye en yakın noktalar.

Dikkat kutuları: öğrencilerin en sık karıştırdığı ayrımlar.

Her bölüm sonundaki çözümlü soruları önce cevaba bakmadan çözmeyi deneyin. Özellikle **makine dili (op-code) çözümleme** ve **maske soruları** hesap gerektirir, pratik yapın.

BÖLÜM 1

Veri Depolama *(Data Storage)*

Bilginin bilgisayar içinde nasıl **kodlandığı** ve **saklandığı**: bitler, kapılar, flip-flop, ana bellek, yığın bellek, metin/sayı/resim/ses gösterimi, ikilik sistem, tam sayı ve kesir saklama, sıkıştırma ve iletişim hataları.

1.1 Bitler ve Saklanmaları

> **TANIM**
>
> **Bit** *(Binary Digit)*: 0 ya da 1 değerini alan ikilik basamak. **Bit desenleri** bilgiyi temsil eder: sayılar, metin karakterleri, resimler, ses ve diğerleri.

Boole işlemleri (Boolean Operations)

Bir veya daha fazla doğru/yanlış (1/0) değeri işleyen işlemlerdir. Adını mantık alanının öncüsü matematikçi **George Boole**'dan (1815–1864) alır. Temel işlemler: **AND, OR, XOR** (dışlayan veya) ve **NOT**.

Girdi A	Girdi B	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

- **AND**: Yalnızca **iki girdi de 1** ise çıktı 1.
- **OR**: **En az bir girdi 1** ise çıktı 1.
- **XOR**: Girdiler **farklı** ise çıktı 1.
- **NOT**: Tek girdilidir, girdiyi **tersine çevirir** (0→1, 1→0).
- **NAND** = AND + NOT. Yalnızca iki girdi 1 ise 0 verir.
- **NOR** = OR + NOT. Yalnızca iki girdi 0 ise 1 verir.

Kapılar (Gates)

Kapı, bir Boole işlemini hesaplayan aygıttır. Genellikle küçük elektronik devrelerle yapılır ve bilgisayarların **yapı taşlarını** oluşturur. **VLSI** *(Very Large Scale Integration, çok büyük ölçekli tümleşim)* teknolojisi, milyonlarca kapı ve flip-flop'un denetim devreleriyle birlikte tek bir yongaya (chip) yerleştirilmesini sağlar.

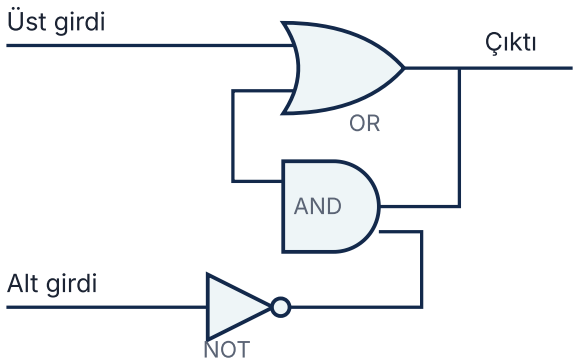


Şekil 1.2 — AND, OR, XOR ve NOT kapılarının sembolleri

Flip-flop

Flip-flop, kapılardan yapılmış ve **bir bit saklayabilen** devredir. Çıktısı 0 ya da 1'dir ve başka bir devreden gelen bir **darbe** (geçici olarak 1 olup tekrar 0'a dönen sinyal) gelene kadar sabit kalır.

- **Üst girdi** geçici olarak 1 yapılırsa çıktı **1** olur.
- **Alt girdi** geçici olarak 1 yapılırsa çıktı **0** olur.
- **İki girdi de 0** iken en son saklanan değer **korunur**.



Şekil 1.3 — Basit bir flip-flop devresi

SINAV · FLİP-FLOP'U ÖĞRETMENİN 3 AMACI

1. Aygıtların kapılardan nasıl yapılabildiğini gösterir. Bu sürece **sayısal devre tasarımı** denir ve bilgisayar mühendisliğinin önemli bir konusudur.
2. **Soyutlama** *(abstraction)* ve soyut araç kullanımına örnektir. Flip-flop farklı iç yapılarla da kurulabilir (Şekil 1.5) ama dış özellikleri aynıdır. Mühendis iç devreyi bilmeden yalnızca dış davranışını bilerek onu kullanabilir.
3. Modern bilgisayarda **bir bit saklamanın** yollarından biridir.

Onaltılık gösterim (Hexadecimal Notation)

Uzun bit desenleri için **kısa gösterimdir**. Desen **dörder bitlik gruplara** ayrılır ve her grup tek bir sembolle gösterilir.

- `10100011` → **`A3`**
- `1010010011001000` → **`A4C8`**

DİKKAT

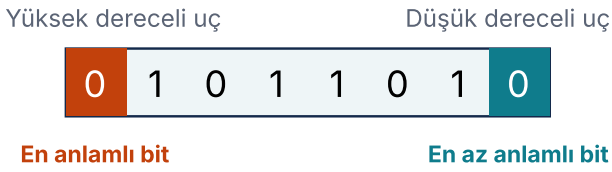
Her hex basamak tam olarak **4 bit**tir. Bit sayısı 4'ün katı değilse dönüştürürken **soldan** 0 ekleyin.

Bit	Hex	Bit	Hex
0000	0	1000	8
0001	1	1001	9
0010	2	1010	A
0011	3	1011	B
0100	4	1100	C
0101	5	1101	D
0110	6	1110	E
0111	7	1111	F

1.2 Ana Bellek (Main Memory)

Bilgisayar, veri saklamak için her biri tek bir bit tutabilen çok sayıda devre (ör. flip-flop) içerir. Bu bit deposuna **ana bellek** denir.

- **Hücre** *(cell)*: Ana belleğin birimi. Genellikle **8 bit = 1 bayt**.
- **En anlamlı bit** *(most significant bit)*: Hücrenin **sol** (yüksek dereceli) ucundaki bit.
- **En az anlamlı bit** *(least significant bit)*: **Sağ** (düşük dereceli) uçtaki bit.
- **Adres** *(address)*: Her hücreyi benzersiz şekilde tanımlayan "ad". Adresler aslında **sıfırdan başlayan ardışık sayılardır**. Bu numaralandırma hücrelere bir **sıra** kazandırır.



Şekil 1.7 — Bayt boyutlu bir bellek hücresinin düzeni

- **RAM** *(Random Access Memory, rastgele erişimli bellek)*: Hücrelerine **herhangi bir sırayla** kolayca erişilebilen bellek.
- Modern RAM'ler çoğunlukla flip-flop yerine bitleri hızla boşalan **küçük elektrik yükleri** olarak saklayan teknolojilerle yapılır. Bu yüzden yükleri saniyede birçok kez yenileyen bir **yenileme devresi** *(refresh circuit)* gerekir.
- Bu uçuculuk nedeniyle bu belleğe **dinamik bellek**, yani **DRAM** *(Dynamic RAM)* denir.

SINAV · BELLEK KAPASİTESİ BİRİMLERİ (2'NİN KUVVETLERİ)

Birim	Değer	Örnek
Kilobayt (KB)	2^{10} = 1.024 bayt	3 KB = 3 × 1.024 bayt
Megabayt (MB)	2^{20} = 1.048.576 bayt	3 MB = 3 × 1.048.576 bayt
Gigabayt (GB)	2^{30} = 1.073.741.824 bayt	3 GB = 3 × 1.073.741.824 bayt

Örnek soru: 4 KB bellekte kaç bit vardır? 4 × 1024 × 8 = **32.768 bit**.

1.3 Yığın Bellek (Mass Storage)

Ana belleğe ek olarak bilgisayarlarda **yığın bellek** ya da **ikincil bellek** *(secondary storage)* bulunur: manyetik diskler, CD, DVD, manyetik bantlar ve flash sürücüler.

ANA BELLEĞE GÖRE YIĞIN BELLEK

- **Çevrim içi** (on-line) ya da **çevrim dışı** (off-line) olabilir.
- Genellikle **daha büyük**,
- **daha az uçucu**,
- ve **daha yavaştır**.

TÜRLERİ

- **Manyetik disk**
- **Manyetik bant**
- **Optik sistemler**: CD, DVD, Blu-ray
- **Flash teknolojisi**: flash sürücüler, SD bellek kartı

Manyetik disk

- Veri, dönen disk üzerindeki eş merkezli dairelere, yani **izlere** *(tracks)* kaydedilir. Okuma/yazma kafası bir **erişim kolu** ile izler arasında hareket eder.
- Her iz **sektör** *(sector)* denen küçük yaylara bölünür. Tüm sektörler aynı sayıda bit içerir (tipik olarak 512 bayt ile birkaç KB arası).

- İz ve sektörler diskin fiziksel parçası değildir. **Biçimlendirme** *(formatting / initializing)* ile manyetik olarak işaretlenir. Disk yeniden biçimlendirilebilir ama bu işlem diskteki **tüm bilgiyi siler**.
- Kapasite; **plaka** *(platter)* sayısına ve iz/sektör yoğunluğuna bağlıdır. Yüksek kapasiteli sistemlerde ortak bir mil üzerinde 3–6 plaka bulunur ve veri plakaların iki yüzüne de yazılabilir.

SINAV · DİSK PERFORMANS ÖLÇÜTLERİ

Arama süresi *seek time*	Okuma/yazma kafasının bir izden diğerine gitme süresi.
Dönme gecikmesi *rotation delay / latency*	Diskin bir tam dönüş süresinin **yarısı**. Kafa doğru izdeyken istenen verinin kafanın altına gelmesi için geçen ortalama süre.
Erişim süresi *access time*	**Arama süresi + dönme gecikmesi**.
Aktarım hızı *transfer rate*	Verinin diske yazılma ya da diskten okunma hızı.

Optik sistemler ve flash

- **CD**: 12 cm çapında, şeffaf koruyucu kaplamalı yansıtıcı malzeme. Bilgi yüzeydeki yansıma farklılıklarıyla kaydedilir ve **lazer** ile okunur. Veri, **içten dışa doğru spiral** çizen tek bir ize yazılır. Sektör başına 2 KB (müzikte saniyenin 1/75'i).
- **DVD**: Birden çok yarı saydam katmandan oluşur. Kapasitesi birkaç GB'tır.
- **Blu-ray (BD)**: Kırmızı yerine **mavi-mor lazer** kullanır. DVD'nin **beş katından fazla** kapasite sunar.
- **Flash bellek**: Manyetik ve optik sistemler **fiziksel hareket** (dönen disk, hareketli kafa, lazer) gerektirdiği için yavaştır. Flash bellekte bitler, depolama ortamına doğrudan elektronik sinyal gönderilerek saklanır. Hareketli parça yoktur.

Dosya saklama

- **Dosya** *(file)*: Yığın bellekte saklanan veri birimi. Disk dosyaları sektör sektör işlenir.
- **Mantıksal kayıt**: Verinin doğal bölümlerine karşılık gelir. **Fiziksel kayıt**: Bir sektörün boyutuna karşılık gelir.
- **Tampon** *(buffer)*: Verinin geçici olarak saklandığı bellek alanı. Genellikle veri aktarımının bir adımıdır. Örnek: yazıcıların belgenin henüz basılmamış kısmını tuttuğu bellek.

1.4 Bilginin Bit Desenleriyle Gösterimi

Metin

Her karaktere (harf, noktalama vb.) benzersiz bir bit deseni atanır.

Kod	Bit	Açıklama
ASCII *(American Standard Code for Information Interchange)*	7 bit	İngilizce metindeki sembollerin çoğunu temsil eder (genelde başına 0 eklenip 8 bit saklanır).
ISO genişletmeleri	8 bit	ASCII'nin büyük dil gruplarına uyarlanmış 8 bitlik uzantıları.

Unicode	16 bit	Dünya dillerindeki başlıca sembolleri temsil eder.

Örnek: `01001000 01100101 01101100 01101100 01101111 00101110` = "**Hello.**" (H = 48_{16}, e = 65_{16}, l = $6C_{16}$, o = $6F_{16}$).

Sayılar

- **İkilik gösterim** *(binary notation)*: Sayıyı 2 tabanında bitlerle gösterir.
- Sınırlamalar: **Taşma** *(overflow)*, değer gösterilemeyecek kadar **büyükse** oluşur. **Kesme** *(truncation)*, değer **tam olarak** gösterilemiyorsa oluşur.

Resimler

- **Bit eşlem** *(bit map)* teknikleri: Resim **piksellerden** (picture element) oluşur. Renk, **RGB** (kırmızı-yeşil-mavi) ya da **parlaklık ve renklilik** (luminance & chrominance) ile kodlanır.
- **Vektör** teknikleri: Resim çizgi ve eğrilerle tarif edilir. **Ölçeklenebilir**dir (büyütünce bozulmaz). Örnek: **TrueType** ve **PostScript**.

Ses

- **Örnekleme** *(sampling)*: Ses dalgasının genliği **düzenli aralıklarla** ölçülür ve değerler kaydedilir. Gerçek sesi kaydeder ve yüksek kaliteli kayıtlarda kullanılır.
- Örnek (Şekil 1.14): 0, 1.5, 2.0, 1.5, 2.0, 3.0, 4.0, 3.0, 0 dizisi.
- Uzak mesafe telefon iletişiminde yıllarca **saniyede 8000 örnek** kullanılmıştır.

1.5 İkilik Sistem (The Binary System)

Onluk sistem **10'un kuvvetlerine**, ikilik sistem **2'nin kuvvetlerine** dayanır. Basamak değerleri sağdan sola: 1, 2, 4, 8, 16, 32, ...

İKİLİK → ONLUK (ŞEKİL 1.16)

Her biti basamak değeriyle çarpıp toplayın.
`100101` = 32 + 0 + 0 + 4 + 0 + 1 = **37**

ONLUK → İKİLİK ALGORİTMASI (ŞEKİL 1.17)

1. Değeri 2'ye böl, kalanı yaz.
2. Bölüm 0 olmayana kadar yeni bölümü 2'ye bölmeye devam et.
3. Bölüm 0 olunca kalanları **sağdan sola**, yazıldıkları sırayla diz.

13 → kalanlar 1, 0, 1, 1 → **`1101`**

İkilik toplama

```
0+0 = 0     0+1 = 1
1+0 = 1     1+1 = 10 (elde 1)

   111010   (58)
 +  11011   (27)
 ---------
  1010101   (85)
```

İkilik kesirler

Noktanın sağındaki basamaklar ½, ¼, ⅛ ... değerindedir.

`101.101` = 4 + 0 + 1 + ½ + 0 + ⅛ = **5⅝**

Toplama, onluk sistemdeki gibi noktalar hizalanarak yapılır.

1.6 Tam Sayıları Saklama

- **İkiye tümleyen gösterimi** *(two's complement)*: Tam sayıları göstermenin **en yaygın** yolu.
- **Fazlalık gösterimi** *(excess notation)*: Tam sayıları göstermenin diğer bir yolu.
- İkisinde de **taşma hatası** olabilir.

İkiye tümleyen

- En soldaki bit **işaret bitidir**: 0 = pozitif/sıfır, 1 = negatif.
- 4 bitte aralık: **−8 ile +7** (`1000` ... `0111`). 8 bitte **−128 ile +127** (`10000000` ... `01111111`).
- n bit için: -2^{n-1} ... $2^{n-1} - 1$. 6 bit: −32 ... 31.

SINAV · NEGATİFİNİ ALMA KISA YOLU (ŞEKİL 1.22)

Sağdan sola doğru **ilk 1 dahil** bitleri olduğu gibi kopyala, **kalan bitleri tersine çevir**. `0110` (+6) → `1010` (−6)

4 bit	İkiye tümleyen	Fazlalık 8
0111	7	-1
0110	6	-2
0101	5	-3
0100	4	-4
0011	3	-5
0010	2	-6
0001	1	-7
0000	0	-8
1111	-1	7
1110	-2	6
1101	-3	5
1100	-4	4
1011	-5	3
1010	-6	2
1001	-7	1
1000	-8	0

İkiye tümleyende toplama ve taşma

Toplama, ikilik toplamayla aynıdır. Soldaki taşan **elde atılır**. **Çıkarma**, çıkan sayının negatifini toplamaya dönüştürülerek yapılır: 7 − 5 = 7 + (−5).

Onluk	İkiye tümleyen	Sonuç
3 + 2	0011 + 0010 = 0101	5
-3 + -2	1101 + 1110 = 1011	-5
7 + -5	0111 + 1011 = 0010	2

DİKKAT · TAŞMA NASIL ANLAŞILIR?

İki pozitif sayı toplanıp sonuç **negatif** çıkarsa ya da **iki negatif** toplanıp sonuç **pozitif** çıkarsa **taşma** vardır. Örnek (4 bit): 5 + 4 = `0101 + 0100 = 1001` → −7 (yanlış!).

Fazlalık gösterimi (Excess notation)

Her değer, ikilik karşılığından sabit bir sayı (fazlalık) çıkarılarak okunur. 4 bitte **fazlalık 8**: `1000` = 0, `1111` = 7, `0000` = −8. 3 bitte **fazlalık 4**: `100` = 0, `111` = 3, `000` = −4. İkiye tümleyenden farkı: **işaret biti terstir** (1 =

pozitif).

1.7 Kesirleri Saklama (Kayan Nokta)

Kesirler **kayan nokta** *(floating-point)* gösterimiyle saklanır. Kitaptaki 8 bitlik örnek biçim: **işaret biti** (1 bit) + **üs alanı** *(exponent, 3 bit, fazlalık 4)* + **mantis** *(mantissa, 4 bit)*.

- Örnek: `0 110 1011` → işaret +, üs 110 = +2, mantis .1011 → noktayı 2 sağa kaydır: 10.11 = **2¾**.
- **Normalleştirilmiş biçim**: Mantisin en soldaki biti 1 olacak şekilde saklanır.
- **Yuvarlama (kesme) hatası** *(round-off / truncation)*: Mantis alanı yetmezse bitler kaybolur. 2⅝ = 10.101 için 5 mantis biti gerekir. 4 bitte .1010 saklanır, değer 2½ olur.

1.8 Veri Sıkıştırma

- **Kayıplı** *(lossy)* ve **kayıpsız** *(lossless)* sıkıştırma
- **Tekrar uzunluğu kodlaması** *(run-length)*: Aynı bitin uzun tekrarları "şu bitten şu kadar" şeklinde kodlanır.
- **Frekansa bağlı kodlama** *(Huffman kodları)*: Sık görülen sembollere kısa, nadir sembollere uzun kod verilir.
- **Göreceli kodlama** *(relative / differential)*: Her veri bloğu, öncekiyle arasındaki **fark** olarak kaydedilir (ör. video kareleri).
- **Sözlük kodlaması** *(dictionary)*: Tekrarlayan yapılar sözlükteki bir indeksle gösterilir. **LZW** gibi **uyarlanır sözlük** yöntemleri sözlüğü kodlama sırasında oluşturur.

Biçim	Kullanım alanı
GIF	Çizgi film/karikatür (az renkli resimler)
JPEG	Fotoğraflar
TIFF	Resim arşivleme
MPEG	Video: yüksek çözünürlüklü TV yayını, video konferans
MP3	Ses: **zamansal maskeleme** (yüksek bir sesten hemen sonraki sessiz sesler duyulmaz) ve **frekans maskeleme** (bir frekanstaki ses yakın frekanstaki daha sessiz sesi bastırır)

1.9 İletişim Hataları

- **Eşlik bitleri** *(parity bits)*: Her desene, desendeki 1'lerin sayısı **tek** (odd parity) ya da **çift** (even parity) olacak şekilde ek bir bit eklenir. Bir bit bozulursa eşlik bozulur ve hata **fark edilir**.
- **Kontrol baytı** *(checkbyte)*: Uzun bir bit dizisi için toplanan eşlik bitlerinden oluşan bayt. Dizinin sonuna eklenir.
- **Hata düzeltme kodları** *(error-correcting codes)*: Hatayı yalnızca tespit etmekle kalmaz, **düzeltir** de. Geçerli kodlar arasında büyük bir **Hamming uzaklığı** bırakılarak bozulan desen en yakın geçerli koda eşlenir.

Bölüm 1 — Çözümlü Sorular

1. Hangi girdi bit desenleri devrenin 1 üretmesini sağlar? (Üstteki iki girdi bir XOR kapısına, XOR çıktısı ve alttaki üçüncü girdi bir AND kapısına gidiyor.)

Cevap: Üstteki iki girdiden **yalnızca biri** 1 olmalı ve en alttaki girdi **1** olmalı.

2. Şekil 1.3'teki flip-flop'ta üst girdi 0'da tutulurken alt girdiye 1 verilirse ne olur?

Cevap: Alt girdideki 1, NOT kapısıyla 0'a çevrilir ve AND kapısının çıktısı 0 olur. OR kapısının iki girdisi de 0 olduğu için çıktısı 0 olur. Bu da AND çıktısının, alt girdi 0'a döndükten sonra da 0 kalmasını sağlar. Flip-flop artık 0 saklar.

3. Şekil 1.5'teki flip-flop'ta iki girdi 0'dayken üst girdi geçici olarak 1 yapılırsa ne olur?

Cevap: Üst OR kapısının çıktısı 1 olur, üst NOT 0 üretir. Alt OR 0 verir, alt NOT 1 üretir. Bu 1 hem çıktıdır hem de üst OR'a geri beslenir. Böylece girdi 0'a dönse bile çıktı 1'de kalır.

4. a) İki girdisi NOT'tan geçirilip NAND kapısına giren devre hangi işlemi yapar? b) Bir AND ve iki NOR kapısından oluşan devre?

Cevap: a) Tüm devre tek bir **OR** kapısına eşdeğerdir. b) Tüm devre tek bir **XOR** kapısına eşdeğerdir.

5. Hex ile gösterin: a) 0110101011110010 b) 111010000101010100010111 c) 01001000

Cevap: a) **6AF2** b) **E85517** c) **48**

6. Şu hex desenler hangi bitleri gösterir? a) 5FD97 b) 610A c) ABCD d) 0100

Cevap: a) `01011111110110010111` b) `0110000100001010` c) `1010101111001101` d) `0000000100000000`

7. 5 numaralı hücrede 8 varsa, 6. hücreye 5 değerini yazmak ile 5. hücrenin içeriğini 6. hücreye taşımak arasındaki fark nedir?

Cevap: İlkinde 6. hücrede **5**, ikincisinde **8** olur.

8. 2 ve 3 numaralı hücrelerin değerlerini değiştirmek için "2'yi 3'e taşı, sonra 3'ü 2'ye taşı" adımları neden yanlıştır? Doğru yöntem nedir?

Cevap: 1. adım 3. hücredeki asıl değeri siler, bu yüzden iki hücre de 2'nin değeriyle kalır. Doğrusu bir ara hücre kullanmaktır: 2 → 1, 3 → 2, 1 → 3.

9. 4 KB belleği olan bilgisayarda kaç bit vardır?

Cevap: 4 × 1024 × 8 = **32.768 bit**.

10. 8 bitlik ASCII ile kodlanmış mesaj: 01000011 01101111 01101101 01110000 01110101 01110100 01100101 01110010 00100000 01010011 01100011 01101001 01100101 01101110 01100011 01100101

Cevap: "**Computer Science**"

11. ASCII'de bir harfin büyük ve küçük hâlinin kodları arasında nasıl bir ilişki vardır?

Cevap: Kodlar aynıdır. Tek fark, düşük dereceli uçtan **6. bit** büyük harfte 0, küçük harfte 1'dir. (Bölüm 2'deki AND 11011111 / OR 00100000 maskelerinin nedeni budur.)

12. Üç baytta, her basamak bir ASCII deseniyle kodlanırsa gösterilebilecek en büyük sayı nedir? İkilik gösterim kullanılırsa?

Cevap: ASCII ile 3 sembol yani **999**. İkilik ile $2^{24} - 1$ = **16.777.215**.

13. İkilik → onluk: a) 0101 b) 1001 c) 1011 d) 0110 e) 10000 f) 10010 · ayrıca 101010, 100001, 10111, 0110, 11111

Cevap: a) 5 b) 9 c) 11 d) 6 e) 16 f) 18 · 42, 33, 23, 6, 31

14. Onluk → ikilik: a) 6 b) 13 c) 11 d) 18 e) 27 f) 4 · ayrıca 32, 64, 96, 15, 27

Cevap: `110, 1101, 1011, 10010, 11011, 100` · `100000, 1000000, 1100000, 1111, 11011`

15. İkilik kesir → onluk: a) 11.01 b) 101.111 c) 10.1 d) 110.011 e) 0.101

Cevap: a) 3¼ b) 5⅞ c) 2½ d) 6⅜ e) ⅝

16. İkilik gösterin: a) 4½ b) 2¾ c) 1⅛ d) 5/16 e) 5⅝

Cevap: a) `100.1` b) `10.11` c) `1.001` d) `0.0101` e) `101.101`

17. İkilik toplama: a) 11011 + 1100 b) 1010.001 + 1.101 c) 11111 + 0001 d) 111.11 + 00.01

Cevap: a) `100111` b) `1011.110` c) `100000` d) `1000.00`

18. İkiye tümleyen → onluk: a) 00011 b) 01111 c) 11100 d) 11010 e) 00000 f) 10000

Cevap: a) 3 b) 15 c) −4 d) −6 e) 0 f) −16

19. Onluk → 8 bitlik ikiye tümleyen: a) 6 b) −6 c) −17 d) 13 e) −1 f) 0

Cevap: a) `00000110` b) `11111010` c) `11101111` d) `00001101` e) `11111111` f) `00000000`

20. Şu ikiye tümleyen değerlerin negatifini bulun: a) 00000001 b) 01010101 c) 11111100 d) 11111110 e) 00000000 f) 01111111

Cevap: a) `11111111` b) `10101011` c) `00000100` d) `00000010` e) `00000000` f) `10000001`

21. İkiye tümleyende 4, 6 ve 8 bitle saklanabilecek en büyük ve en küçük sayılar?

Cevap: 4 bit: 7 ve −8 · 6 bit: 31 ve −32 · 8 bit: 127 ve −128

22. İkiye tümleyende toplayın: a) 0101+0010 b) 0011+0001 c) 0101+1010 d) 1110+0011 e) 1010+1110

Cevap: a) `0111` (5+2=7) b) `0100` (3+1=4) c) `1111` (5+(−6)=−1) d) `0001` (−2+3=1) e) `1000` (−6+(−2)=−8)

23. Taşmaya dikkat ederek toplayın: a) 0100+0011 b) 0101+0110 c) 1010+1010 d) 1010+0111 e) 0111+0001

Cevap: a) `0111` b) `1011` (**taşma**) c) `0100` (**taşma**) d) `0001` e) `1000` (**taşma**)

24. 4 bitlik ikiye tümleyene çevirip toplamaya dönüştürün: a) 6−(−1) b) 3−2 c) 4−6 d) 2−(−4) e) 1−5

Cevap: a) `0110+0001=0111` b) `0011+1110=0001` c) `0100+1010=1110` d) `0010+0100=0110` e) `0001+1011=1100`

BÖLÜM 2

Veri İşleme *(Data Manipulation)*

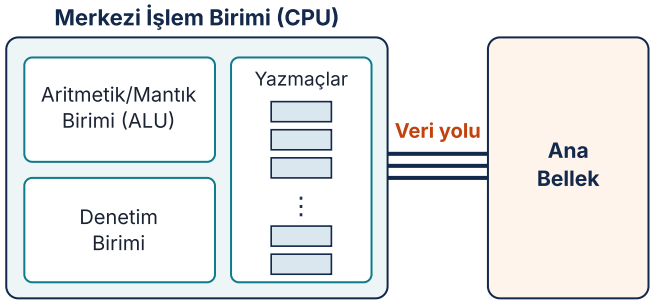
Bölüm 1'de verinin nasıl *saklandığını* gördük. Bu bölüm, bilgisayarın veriyi nasıl *işlediğini*, yani veriyi bir yerden başka bir yere taşımasını ve üzerinde aritmetik/mantık işlemleri yapmasını anlatır.

2.1 Bilgisayar Mimarisi

Veriyi işleyen devreye **Merkezi İşlem Birimi** *(CPU, Central Processing Unit)* ya da kısaca **işlemci** denir. Günümüz CPU'ları, ana devre kartına *(motherboard)* takılan küçük, düz kareler şeklindedir. Boyutları küçük olduğu için **mikroişlemci** *(microprocessor)* olarak adlandırılırlar.

CPU'nun üç parçası

Birim	Görevi
Aritmetik/Mantık Birimi *ALU*	Veri üzerinde işlem yapan devreler (toplama, çıkarma, AND, OR...).
Denetim Birimi *Control Unit*	Makinenin faaliyetlerini koordine eden devreler.
Yazmaç Birimi *Register Unit*	CPU içinde geçici bilgi tutan hücreler: **yazmaçlar** (registers).



Şekil 2.1 — CPU ve ana bellek bir veri yolu (bus) ile bağlıdır.

Yazmaçlar (Registers)

- **Genel amaçlı yazmaçlar** *(general-purpose)*: CPU'nun üzerinde çalıştığı verilerin geçici tutulduğu yerlerdir. ALU'nun **girdilerini** tutar ve **sonuçlarını** saklar.
- **Özel amaçlı yazmaçlar** *(special-purpose)*: Belirli bir görevi vardır. Örnek: **Program Sayacı** (PC) ve **Komut Yazmacı** (IR) (bkz. 2.3).

Bellekteki bir veri üzerinde işlem yapmak için denetim birimi: (1) veriyi bellekten genel amaçlı yazmaçlara aktarır, (2) ALU'ya verinin hangi yazmaçlarda olduğunu bildirir, (3) ALU'daki uygun devreyi etkinleştirir, (4) sonucun hangi yazmaca yazılacağını söyler.

Veri Yolu (Bus)

CPU ile ana bellek, bit desenlerini aktarmak için **veri yolu** denen bir kablo demetiyle bağlıdır.

- **Okuma (read):** CPU, ilgili bellek hücresinin **adresini** ve "oku" sinyalini gönderir.
- **Yazma (write):** CPU, hedef hücrenin **adresini**, **saklanacak veriyi** ve "yaz" sinyalini gönderir.

TANIM · BELLEKTE SAKLANAN İKİ DEĞERİ TOPLAMA (ŞEKİL 2.2)

1. Değerlerden birini bellekten al, bir yazmaca koy.
2. Diğer değeri bellekten al, başka bir yazmaca koy.
3. Toplama devresini bu iki yazmaçla çalıştır, sonucu üçüncü bir yazmaca yaz.
4. Sonucu belleğe kaydet.
5. Dur.

TANIM · BÖLME ÖRNEĞİ (ŞEKİL 2.3)

1. Bir değeri bellekten yazmaca yükle (LOAD).
2. Diğer değeri yükle.
3. **İkinci değer sıfırsa 6. adıma atla** (JUMP). Bu, **koşullu atlamadır** ve sıfıra bölmeyi engeller.
4. Böl, sonucu üçüncü yazmaca koy.
5. Sonucu belleğe kaydet (STORE).
6. Dur.

Bellek hiyerarşisi ve önbellek (Cache Memory)

Bellek	Ne tutar?	Hız
Yazmaçlar	O anda üzerinde işlem yapılan veriyi	En hızlı
Önbellek (cache)	Ana belleğin o an ilgilenilen kısmının kopyasını (birkaç yüz KB, CPU içinde)	Çok hızlı
Ana bellek	Yakın gelecekte gerekecek veriyi	Orta
Yığın bellek (mass storage)	Yakın gelecekte büyük olasılıkla gerekmeyecek veriyi	En yavaş

Önbellek kullanılınca veri aktarımları yazmaçlar ile önbellek arasında yapılır, değişiklikler ana belleğe toplu olarak uygun bir zamanda aktarılır. Böylece makine döngüsü ana bellek iletişimini beklemeden daha hızlı ilerler.

SINAV · KAYITLI PROGRAM KAVRAMI (STORED PROGRAM CONCEPT)

Bir program bit desenleri olarak kodlanıp **ana bellekte saklanabilir**. CPU komutları buradan alıp yürütür. Bu sayede çalıştırılacak program kolayca değiştirilebilir. Program ile veri **aynı bellekte** durur.

2.2 Makine Dili

TANIM

Makine komutu *(machine instruction)*: CPU'nun tanıyabildiği, bit deseni olarak kodlanmış bir komut.
Makine dili *(machine language)*: Bir makinenin tanıdığı komutların **tamamı**.

Makine dili felsefeleri: RISC ve CISC

	RISC (Reduced Instruction Set Computing)	**CISC** (Complex Instruction Set Computing)
Komut sayısı	Az	Çok
Komutların yapısı	Basit, verimli ve hızlı	Kullanışlı ve güçlü (karmaşık)

Örnek	PowerPC (Apple/IBM/Motorola), **ARM**	**Intel**

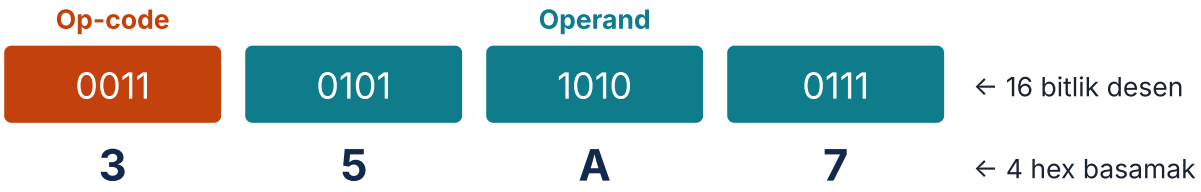
Makine komutu türleri

Tür	Açıklama	Örnekler
Veri aktarımı *Data Transfer*	Veriyi bir yerden başka bir yere **kopyalar**. Toplama örneğinde 1, 2 ve 4. adımlar.	**LOAD**: bellek hücresi → yazmaç **STORE**: yazmaç → bellek hücresi
Aritmetik/Mantık *Arithmetic/Logic*	Denetim birimine ALU'da işlem yaptırır. Toplama örneğinde 3. adım.	ADD, AND, OR, XOR, SHIFT, ROTATE
Denetim *Control*	Veriyi değil, **programın akışını** yönetir. Toplama örneğinde 5. adım.	**JUMP / BRANCH**, HALT

> **DİKKAT**
>
> LOAD ve STORE yönlerini karıştırmayın: **LOAD** yazmacı bellekten *doldurur*, **STORE** yazmacın içeriğini belleğe *yazar*. JUMP, sıradaki komut yerine **başka bir komutun** çalışmasını sağlar.

Örnek makine (Ek C / Appendix C)

- **16 genel amaçlı yazmaç** (0–F), **256 bellek hücresi** (adres 00–FF), her hücre **8 bit**.
- Toplam **12 temel komut**. Her komut **16 bit = 4 onaltılık (hex) basamak**, yani bellekte **2 hücre** kaplar.
- Komut iki alandan oluşur: **İşlem kodu** *(op-code)* = ilk 4 bit (ilk hex basamak, 1–C) ve **İşlenen** *(operand)* = kalan 12 bit (3 hex basamak, işlemin ayrıntıları).



35A7 = "5 numaralı yazmacın içeriğini A7 adresli bellek hücresine STORE et."

Şekil 2.5 / 2.6 — Bir komutun bileşimi ve 35A7 komutunun çözümlenmesi

Ek C komut tablosu *(sınavda en çok kullanılacak tablo)*

R, S, T = yazmaç numaraları; X, Y = hex basamaklar; XY = bellek adresi veya sabit değer.

Op-code	Biçim	Anlamı	Örnek
1	`1RXY`	**LOAD**: R yazmacına **XY adresindeki** bellek hücresinin içeriğini yükle	`14A3 → A3 hücresi → R4`
2	`2RXY`	**LOAD**: R yazmacına **XY değerinin kendisini** yükle (sabit)	`20A3 → R0 = A3`
3	`3RXY`	**STORE**: R yazmacının içeriğini XY adresli hücreye yaz	`35B1 → R5 → B1 hücresi`
4	`40RS`	**MOVE**: R yazmacının içeriğini S yazmacına kopyala	`40A4 → RA → R4`
5	`5RST`	**ADD** (ikiye tümleyen): S + T → R	`5726 → R2 + R6 → R7`

6	`6RST`	**ADD** (kayan nokta): S + T → R	`634E → R4 + RE → R3`
7	`7RST`	**OR**: S OR T → R	`7CB4 → RB OR R4 → RC`
8	`8RST`	**AND**: S AND T → R	`8045 → R4 AND R5 → R0`
9	`9RST`	**XOR**: S XOR T → R	`95F3 → RF XOR R3 → R5`
A	`AR0X`	**ROTATE**: R yazmacını **sağa** X bit döndür	`A403 → R4 sağa 3 bit`
B	`BRXY`	**JUMP**: R yazmacı **R0'a eşitse** XY adresindeki komuta atla. (B0XY = koşulsuz atlama)	`B43C → R4 = R0 ise 3C'ye`
C	`C000`	**HALT**: yürütmeyi durdur	`C000`

DİKKAT · 1RXY İLE 2RXY FARKI (SIK SORULUR)

`15AB`: CPU belleğe gider, **AB adresindeki** değeri okur ve R5'e koyar (bellek erişimi var).
`25AB`: Bellek erişimi yoktur. **AB değerinin kendisi** R5'e konur.

2.3 Program Yürütme

Bilgisayar, bellekte saklı programı komutları gerektikçe CPU'ya kopyalayarak izler. Komutlar bellekteki sırayla getirilir; bu sıra yalnızca bir **JUMP** komutuyla değişir. İki özel amaçlı yazmaç kritik rol oynar:

KOMUT YAZMACI · INSTRUCTİON REGİSTER (IR)

O anda **yürütülen komutu** tutar.

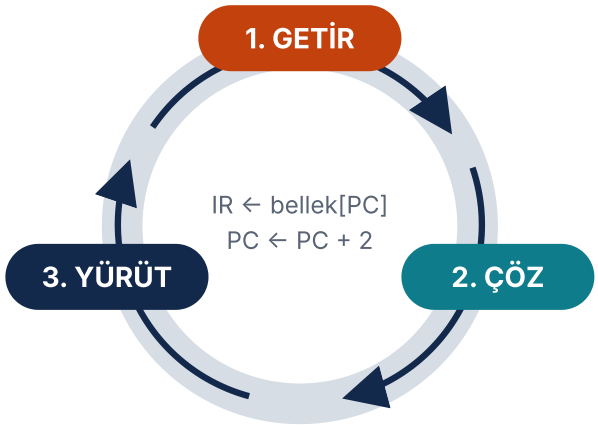
PROGRAM SAYACI · PROGRAM COUNTER (PC)

Bir sonraki yürütülecek komutun **adresini** tutar. Makinenin programda nerede olduğunu izler.

Makine Döngüsü (Machine Cycle)

1. **Getir** *(Fetch)*: PC'nin gösterdiği adresteki komutu bellekten al (komut 2 bayt olduğundan **2 hücre** okunur), IR'ye koy ve **PC'yi 2 artır**.
2. **Çöz** *(Decode)*: IR'deki komutun operand alanını, op-code'a göre parçalarına ayır.
3. **Yürüt** *(Execute)*: Uygun devreyi etkinleştirerek istenen işlemi yap (ör. bellekten yükle veya ALU'da topla).

Ardından döngü yeniden **getir** adımıyla başlar. PC bir önceki getirme adımında artırıldığı için doğru adresi gösterir.



Şekil 2.8 — Makine döngüsü

SINAV · JUMP KOMUTUNUN YÜRÜTÜLMESİ (B258)

`B258` = "R2'nin içeriği R0'a **eşitse** 58 adresindeki komuta atla." Yürütme adımında R2 ile R0 karşılaştırılır:

- **Farklıysa**: yürütme biter, sıradaki komutla yeni döngü başlar.
- **Eşitse**: PC'ye **58** yazılır, bir sonraki getirme 58 adresinden yapılır.

Adım adım örnek: Şekil 2.7 programı (A0 adresinden başlıyor)

Program, bellekteki iki değeri (6C ve 6D) toplayıp sonucu 6E'ye yazar. Başlangıçta PC = A0.

Döngü	Getirilen adres	IR	Getirmeden sonra PC	Yürütme (anlam)
1	`A0-A1`	**`156C`**	`A2`	R5 ← bellek[6C]
2	`A2-A3`	**`166D`**	`A4`	R6 ← bellek[6D]
3	`A4-A5`	**`5056`**	`A6`	R0 ← R5 + R6 (ikiye tümleyen toplama)
4	`A6-A7`	**`306E`**	`A8`	bellek[6E] ← R0
5	`A8-A9`	**`C000`**	`AA`	HALT: makine yürütme adımında durur

DİKKAT · PROGRAM İLE VERİ AYRIMI

Veri de program da bellekte 0 ve 1'lerden oluşur. **Makine tek başına hangisinin veri, hangisinin program olduğunu bilemez.** PC'ye yanlışlıkla bir veri adresi verilirse CPU o veriyi komutmuş gibi çözüp yürütür. Sonuç, verinin içeriğine bağlıdır.

EK BİLGİ · BİLGİSAYAR GÜCÜNÜ KARŞILAŞTIRMA

Saat *(clock)*, faaliyetleri koordine eden darbeler üretir. Hızı **Hz** ile ölçülür (MHz = milyon, GHz = 1000 MHz). Farklı CPU'lar bir saat döngüsünde farklı miktarda iş yaptığından tek başına saat hızı yanıltıcıdır. Daha anlamlı karşılaştırma, aynı programın farklı makinelerde çalıştırılmasıyla yapılır: **kıyaslama** *(benchmarking)*.

2.4 Aritmetik/Mantık Komutları

Mantık işlemleri (bit bit uygulanır)

A	B	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	**1**	1	**0**

```
  10011010        10011010        10011010
AND 11001001    OR  11001001    XOR 11001001
──────────      ──────────      ──────────
  10001000        11011011        01010011
```

SINAV · MASKELEME (MASKİNG): EZBERLENECEK 3 KURAL

Amaç	İşlem	Maskede...	Örnek
Bazı bitleri **0 yapmak** (temizlemek), diğerlerini korumak	**AND**	Sıfırlanacak yere **0**, korunacak yere **1**	`00001111 AND 10101010 = 00001010`
Bazı bitleri **1 yapmak**, diğerlerini korumak	**OR**	1 yapılacak yere **1**, korunacak yere **0**	`11110000 OR 10101010 = 11111010`
Bazı bitleri **tersine çevirmek** (tümleyen)	**XOR**	Çevrilecek yere **1**, korunacak yere **0**	`11111111 XOR 10101010 = 01010101`

Örnek: Bir baytın baştan 3. bitini **0** yapmak için `11011111` ile AND; **1** yapmak için `00100000` ile OR. Tüm baytı tersine çevirmek için `11111111` ile XOR.

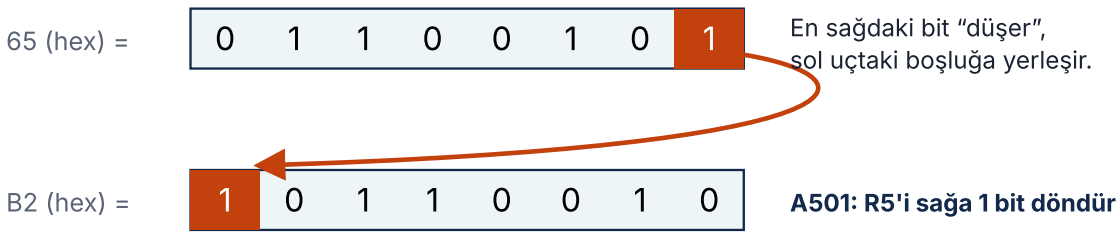
Ek C'de **7 = OR**, **8 = AND**, **9 = XOR**'dur. Örneğin `7ABC`: RB OR RC sonucunu RA'ya koy.

Neden XOR "eldesiz toplama"dır? AND, iki biti çarpma gibi birleştirir. XOR ise toplamaya benzer. Tek fark, iki bit de 1 olduğunda XOR **0** verir, toplama ise **10** (elde 1) verir.

Döndürme ve Kaydırma (Rotation & Shift)

Bitleri bir yazmaç içinde sağa veya sola hareket ettirir. Genellikle hizalama problemlerinde kullanılır. Türleri, dışarı düşen bitin ve oluşan boşluğun ne olduğuna göre ayrılır:

Tür	Dışarı düşen bit ne olur?	Özellik
Dairesel kaydırma = Döndürme *circular shift / rotation*	Diğer uçtaki boşluğa yerleşir	8 bitlik bir deseni 8 kez döndürünce başlangıç desenine dönülür.
Mantıksal kaydırma *logical shift*	Atılır, boşluk her zaman **0** ile doldurulur	**Sola kaydırma = 2 ile çarpma**, **sağa kaydırma = 2'ye bölme**.
Aritmetik kaydırma *arithmetic shift*	İşaret biti değişmeden bırakılır	İşaretli sayılarda işaret korunur.



Şekil 2.12 — 65 (hex) desenini sağa bir bit döndürmek: 01100101 → 10110010 = B2

Ek C makinesinde yalnızca **sağa dairesel kaydırma** vardır (op-code **A**). İlk operand basamağı yazmacı, son basamak döndürülecek bit sayısını belirtir.

DİKKAT · SAĞA/SOLA DÖNÜŞÜM

8 bitlik bir dizide **sağa n bit** döndürmek, **sola (8 − n) bit** döndürmekle aynıdır. Örnek: sağa 3 = sola 5. Bu yüzden Ek C'de sola döndürme gerekirse sağa (8 − n) kullanılır. Örneğin sola 2 için `A506` yazılır.

2.5 Diğer Cihazlarla İletişim

CPU ve ana bellek bilgisayarın çekirdeğidir. Bu çekirdek; yığın bellek, yazıcı, klavye, fare, ekran, kamera ve başka bilgisayarlar gibi **çevre birimleriyle** *(peripheral devices)* iletişim kurar.

Denetleyiciler (Controllers)

- Bilgisayar ile cihaz arasındaki iletişimi **aracı bir aygıt** olan denetleyici yürütür. Ana kart üzerinde kalıcı devre olarak bulunabilir.
- Denetleyici, kasadaki cihazlara kabloyla ya da bilgisayarın arkasındaki **port** denen bağlantı noktasına bağlanır.
- Denetleyiciler, CPU ile ana belleği bağlayan **aynı veri yoluna** bağlıdır (Şekil 2.13). Böylece CPU onlarla belleğe konuşur gibi konuşur.
- **Belleğe eşlemeli G/Ç** *(memory-mapped I/O)*: Cihaza veri göndermek için STORE benzeri, veri almak için LOAD benzeri komut kullanılır.

Doğrudan Bellek Erişimi (DMA)

Denetleyici, CPU'nun veri yolunu kullanmadığı anlarda **ana bellekle doğrudan** iletişim kurabilir. CPU başka işlere devam ederken denetleyici veriyi belleğe yerleştirir, böylece **iki iş aynı anda** yapılır. Performans için önemli bir avantajdır.

El sıkışma (Handshaking)

Veri aktarımı tek yönlü değildir. Bilgisayar ve cihaz, **cihazın durumu** hakkında sürekli iki yönlü bilgi alışverişi yapar. Örneğin bilgisayar karakterleri yazıcının basabileceğinden çok daha hızlı gönderir. El sıkışma olmasa yazıcı geride kalır ve **veri kaybolur**.

İletişim türü	Açıklama
Paralel	Birden çok yol üzerinden bitler **aynı anda** aktarılır.
Seri	Bitler **tek bir yol** üzerinden **art arda** aktarılır.

Birim	Değer
bps	saniyedeki bit
Kbps	1.000 bps
Mbps	1.000.000 bps
Gbps	1.000.000.000 bps

Bant genişliği *(bandwidth)*: Kullanılabilecek **en yüksek** iletişim hızı.

2.6 Diğer Mimariler

Amaç **iş hacmini** *(throughput)* artırmaktır:

- **Boru hattı** *(Pipelining)*: Makine döngüsünün adımlarını **üst üste bindirmek**. Bir komut yürütülürken sonraki çözülür, ondan sonraki getirilir.
- **Paralel işleme** *(Parallel Processing)*: Birden fazla işlemciyi aynı anda kullanmak.

Tür	Açılımı	Anlamı
SISD	Single Instruction, Single Data	Paralel işleme **yok** (klasik tek işlemci).
MIMD	Multiple Instruction, Multiple Data	**Farklı programlar, farklı veriler**. Her işlemci bağımsız çalışır.
SIMD	Single Instruction, Multiple Data	**Aynı program, farklı veriler**. Tüm işlemciler aynı komutu farklı verilere uygular.

Bölüm 2 — Çözümlü Sorular

Slaytlardaki soru ve cevapların Türkçe hâli. Önce kendiniz çözmeye çalışın.

1. Bir bellek hücresinin içeriğini başka bir hücreye taşımak için hangi olaylar gerekir?

Cevap: Bazı makinelerde iki adımlıdır: önce kaynak hücre bir yazmaca okunur, sonra yazmaçtan hedef hücreye yazılır. Çoğu makinede ise ara yazmaç kullanmadan tek işlemle yapılır.

2. Bir bellek hücresine değer yazmak için CPU belleğe hangi bilgileri vermelidir?

Cevap: Yazılacak değer, yazılacak hücrenin adresi ve “yaz” komutu.

3. Yığın bellek, ana bellek ve genel amaçlı yazmaçların kullanım farkı nedir?

Cevap: Yazmaçlar o anki işlem için gerekli veriyi, ana bellek yakın gelecekte gerekecek veriyi, yığın bellek ise yakın gelecekte büyük olasılıkla gerekmeyecek veriyi tutar.

4. Şu Ek C komutlarını Türkçeye çevirin: a) 368A b) BADE c) 803C d) 40F4

Cevap: a) R6'nın içeriğini 8A hücresine STORE et. b) RA'nın içeriği R0'a eşitse DE adresine JUMP et. c) R3 ile RC'yi AND'le, sonucu R0'a koy. d) RF'nin içeriğini R4'e MOVE et.

5. Şu ifadeleri Ek C makine diline çevirin: a) R3'e 56 değerini yükle b) R5'i sağa 3 bit döndür c) RA ile R5'i AND'le, sonucu R0'a koy

Cevap: a) `2356` b) `A503` c) `80A5`

6. Bellek: 00:14, 01:02, 02:34, 03:17, 04:C0, 05:00. PC = 00 ile başlanırsa makine durduğunda 17 adresli hücrede ne vardır?

Cevap: **34**. 1402 komutu 02 hücresindeki 34 değerini R4'e yükler. 3417 komutu R4'ü 17 hücresine yazar. C000 ile makine durur.

7. Bellek B0–B8: 13 B8 A3 02 33 B8 C0 00 0F. PC = B0. a) İlk komuttan sonra R3'te ne var? b) Durduğunda B8 hücresinde ne var?

Cevap: a) `13B8`: B8 hücresindeki **0F** değeri R3'e yüklenir. b) `A302` R3'ü sağa 2 döndürür: 00001111 → 11000011 = **C3**. `33B8` bunu B8'e yazar ve C000 ile durur.

8. Bellek A4–B1: 20 00 21 03 22 01 B1 B0 50 02 B0 AA C0 00. PC = A4. a) AA'daki komut ilk kez çalıştığında R0'da ne var? b) İkinci kez? c) Makine durmadan önce AA'daki komut kaç kez çalışır?

Cevap: a) **00** b) **01** c) **Dört kez**. Program R0'ı 1'er artıran bir döngüdür. R0, R1'deki 03 değerine eşit olunca B1B0 komutu B0'daki HALT'a atlar.

9. İşlemleri yapın: 01001011 AND 10101011 · 10000011 AND 11101100 · 11111111 AND 00101101 · aynıları OR ve XOR ile

Cevap: AND: `00001011, 10000000, 00101101` · OR: `11101011, 11101111, 11111111` · XOR: `11100000, 01101111, 11010010`

10. Bir baytın ortadaki 4 bitini koruyup diğer 4 bitini 0 yapmak için hangi maske ve işlem gerekir?

Cevap: `00111100` maskesi ile **AND**.

11. Ortadaki 4 biti tersine çevirip (tümleyen) diğerlerini korumak için?

Cevap: `00111100` maskesi ile **XOR**.

12. ASCII küçük harfleri büyüğe, büyükleri küçüğe çevirmek için hangi işlem ve maske?

Cevap: Küçük → büyük: `11011111` ile **AND**. Büyük → küçük: `00100000` ile **OR**.

13. 3 bit sağa dairesel kaydırma: a) 01101010 b) 00001111 c) 01111111

Cevap: a) `01001101` b) `11100001` c) `11101111`

14. 1 bit sola dairesel kaydırma (hex): a) AB b) 5C c) B7 d) 35

Cevap: a) **57** b) **B8** c) **6F** d) **6A**

15. 8 bitlik bir dizide 3 bit sağa dairesel kaydırma, kaç bit sola kaydırmaya eşittir?

Cevap: **5** (8 − 3).

16. A7 hücresinin en anlamlı bitini 1 yapan, diğer bitleri değiştirmeyen Ek C programını yazın.

Cevap: `12A7` (R2 ← [A7]) · `2380` (R3 ← 80) · `7023` (R0 ← R2 OR R3) · `30A7` ([A7] ← R0) · `C000` (HALT)

17. E0 hücresinin ortadaki 4 bitini E1'in düşük 4 bitine kopyalayan, E1'in yüksek 4 bitini 0 yapan program?

Cevap: `15E0` (R5 ← [E0]) · `A502` (R5'i sola 2 = sağa 6 döndür) · `260F` (R6 ← 0F) · `8056` (R0 ← R5 AND R6) · `30E1` ([E1] ← R0) · `C000`

18. Ek C makinesi belleğe eşlemeli G/Ç kullanıyor ve yazıcı portu B5. a) R7'deki "A" harfini yazdıran komut? b) Saniyede 1 milyon komut çalışırsa bu karakter saniyede kaç kez gönderilebilir? c) Dakikada 5 sayfa basan yazıcı buna yetişir mi?

Cevap: a) `37B5` b) Bir milyon kez. c) **Hayır.** Bir sayfa 4000 karakterden azdır, yani yazıcı dakikada en fazla ~20.000 karakter basar. Bu, saniyede bir milyon karakterin çok altındadır. Bu yüzden yazıcının bilgisayara "bekle" diyebilmesi, yani **el sıkışma**, gerekir.

19. Disk dakikada 3000 devir yapıyor, her izde 16 sektör, her sektörde 1024 bayt var. Denetleyicinin yetişmesi için gereken iletişim hızı yaklaşık nedir?

Cevap: Saniyede 50 devir × 16 = saniyede 800 sektör. 800 × 1024 bayt × 8 ≈ **6,5 Mbps**.

BÖLÜM 3

İşletim Sistemleri *(Operating Systems)*

İşletim sistemi, bilgisayarın **genel işleyişini denetleyen yazılımdır**. Dosya saklama ve geri alma imkânı, programları çalıştırma isteği için bir arayüz ve programların çalışması için gerekli ortamı sağlar.

Örnek işletim sistemleri

- **Windows**: Microsoft. PC dünyasında yaygın.
- **UNIX**: Büyük sistemlerde ve PC'lerde popüler. **Mac OS** (Apple) ve **Solaris** (Sun, şimdi Oracle) UNIX çekirdeklidir.
- **Linux**: Başlangıçta ticari olmayan şekilde geliştirildi.

Sıradan kullanıcı için farklar çoğunlukla görseldir. Profesyoneller için ise araç ve çalışma felsefesinde büyük farklar demektir.

TANIM · İŞLETİM SİSTEMİNİN İŞLEVLERİ

- Bilgisayarın işleyişini **gözetmek**
- Dosyaları **saklamak ve geri getirmek**
- Programları yürütme için **zamanlamak** (schedule)
- Programların yürütülmesini **koordine etmek**

3.1 İşletim Sistemlerinin Tarihçesi *(Paylaşımlı hesaplamanın evrimi)*

Aşama	Açıklama
Toplu işleme *Batch processing*	İşler (program + veri + yönergeler) yığın bellekte bir **iş kuyruğunda** *(job queue)* bekler. Kuyruk **FIFO**'dur (ilk giren ilk çıkar). Çoğu sistem **öncelik** de tanır: yüksek öncelikli bir iş, bekleyen işin önüne geçebilir. Kullanıcı yürütme sırasında programla etkileşmez.
Etkileşimli işleme *Interactive processing*	Çalışan program, uzak terminaller üzerinden kullanıcıyla **diyalog** kurar. Bu, **gerçek zamanlı işleme** gerektirir.
Gerçek zamanlı işleme *Real-time processing*	Görevlerin, ortamdaki **son tarihlere (deadline) uygun** olarak yerine getirilmesi.
Zaman paylaşımı *Time-sharing*	**Birden çok kullanıcıya aynı anda** hizmet. **Çoklu programlama** ile uygulanır. İlk sistemler ~30 kullanıcıya kabul edilebilir gerçek zamanlı hizmet verebiliyordu.
Çoklu programlama *Multiprogramming*	Zaman aralıklara bölünür, her iş bir aralıkta çalışır. Aralık sonunda iş kenara alınır, sıradaki çalışır. Böylece işlerin **aynı anda çalıştığı yanılsaması** oluşur.
Çok görevlilik *Multitasking*	Aynı tekniğin **tek kullanıcılı** sistemdeki adı. Bir kullanıcı birçok görevi aynı anda yürütür.
Çok işlemcili makineler *Multiprocessor*	Görevler farklı işlemcilere dağıtılır. İki sorunla uğraşılır: **yük dengeleme** (load balancing), yani görevleri işlemcilere dinamik dağıtarak hepsini verimli kullanmak, ve **ölçekleme** (scaling), yani görevi işlemci sayısına uygun alt görevlere bölmek.

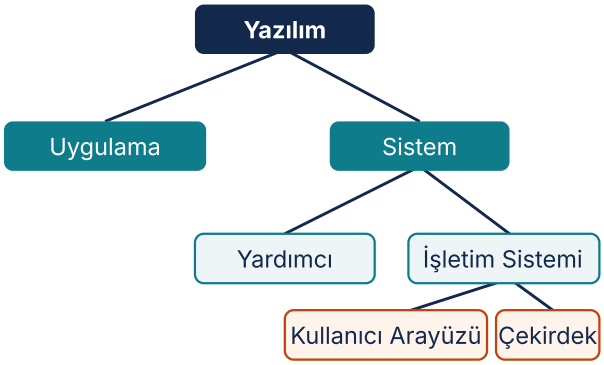
DİKKAT · ZAMAN PAYLAŞIMI İLE ÇOK GÖREVLİLİK FARKI

Zaman paylaşımı: birden fazla kullanıcı aynı makineye aynı anda erişir. **Çok görevlilik:** bir kullanıcı aynı anda birden fazla görev yürütür.

3.2 İşletim Sistemi Mimarisi

Yazılım türleri

- **Uygulama yazılımı** *(application)*: Kullanıcı için belirli görevleri yapar. Örnek: hesap tablosu, veritabanı, masaüstü yayıncılık, muhasebe, program geliştirme yazılımı, oyunlar.
- **Sistem yazılımı** *(system)*: Uygulama yazılımına **altyapı** sağlar. **İşletim sistemi** ve **yardımcı yazılımlardan** *(utility)* oluşur. Yardımcı yazılım örnekleri: veri sıkıştırma/açma, multimedya oynatma, ağ iletişimi.



Şekil 3.3 — Yazılım sınıflandırması

Uygulama ile yardımcı yazılım arasındaki çizgi belirsizdir. Kabaca: **yardımcı yazılım temel, evrensel görevleri**, **uygulama yazılımı ise o makinenin kullanım amacına özgü görevleri** yapar.

İşletim sistemi bileşenleri

KULLANICI ARAYÜZÜ · USER INTERFACE

Kullanıcılarla iletişim kurar. Kullanıcılar ile çekirdek arasında **aracıdır** (Şekil 3.4).

- **Metin tabanlı**: **Kabuk** *(shell)*. Klavye ve ekranda metin mesajlarıyla çalışır.
- **Grafik kullanıcı arayüzü** *(GUI)*.

ÇEKİRDEK · KERNEL

İşletim sisteminin **iç kısmıdır**. Temel işlevleri yerine getirir:

- Dosya yöneticisi *(file manager)*
- Aygıt sürücüleri *(device drivers)*
- Bellek yöneticisi *(memory manager)*
- Zamanlayıcı ve dağıtıcı *(scheduler & dispatcher)*

Çekirdek bileşeni	Görevi (tek cümlede ezberleyin)
Dosya yöneticisi	Yığın belleğin kullanımını koordine eder. Dosyaların yerini, kimin erişebileceğini ve boş alanları kaydeder. **Dizin/Klasör** (directory/folder): kullanıcının oluşturduğu dosya ve alt dizin grubu. **Dizin yolu** (path): iç içe dizinler dizisi.
Aygıt sürücüleri	Çevre birimlerinin denetleyicileriyle iletişim kurar. Her biri kendi cihazına özeldir ve genel istekleri o cihazın teknik adımlarına çevirir.
Bellek yöneticisi	Ana belleğin kullanımını koordine eder. Gerektiğinde **sayfalama** ile **sanal bellek** oluşturur.
Zamanlayıcı	Hangi faaliyetlerin yürütülmek üzere ele alınacağını belirler (süreç tablosuna ekler/çıkarır).
Dağıtıcı	Bu faaliyetlere **CPU zamanının** tahsisini denetler.

SINAV · SAYFALAMA VE SANAL BELLEK (PAGİNG & VİRTUAL MEMORY)

Gereken bellek, fiziksel ana bellekten büyükse bellek yöneticisi programları ve verileri ana bellek ile yığın bellek (disk) arasında gidip getirerek **ek bellek yanılsaması** yaratır. Buna **sayfalama** denir. **Örnek:** 8 GB gerekiyor, makinede 4 GB var. Diskte 4 GB ayrılır. Veri, **sayfa** denen birkaç KB'lık eşit birimlere bölünür. O an gereken sayfalar ana bellekte tutulur, diğerleri diske gönderilir. Bilgisayar 8 GB'ı varmış gibi çalışır. Bu "hayali" büyük bellek alanına **sanal bellek** denir.

Önyükleme (Bootstrapping / Booting)

Bilgisayar her açıldığında işletim sistemini **yığın bellekten** (kalıcı olarak saklandığı yer) **ana belleğe** (açılışta boştur) aktaran işlemdir.

- CPU, her açılışta PC'nin **önceden belirlenmiş bir adresle** başlayacağı şekilde tasarlanır.
- Ana bellek genelde **uçucudur** *(volatile)*: güç kesilince içeriği silinir.
- Çözüm: Belleğin CPU'nun ilk programı aradığı küçük bir kısmı **uçucu olmayan** hücrelerden yapılır. Bu **ROM** (Read-Only Memory, salt okunur bellek) içeriği okunabilir ama değiştirilemez.
- ROM'da saklı programa **önyükleyici** *(boot loader)* denir. Bu, bir **firmware** (donanım yazılımı) örneğidir.

ÖNYÜKLEME ADIMLARI (ŞEKİL 3.5)

1. Makine açılır, CPU **ROM'daki önyükleyiciyi** çalıştırır. İşletim sistemi yığın bellekte durur.
2. Önyükleyici, CPU'ya işletim sistemini yığın bellekten (veya flash bellekten) **ana belleğin uçucu kısmına** aktarmasını söyler.
3. Önyükleyici, ana bellekteki işletim sistemine bir **JUMP** komutu yaptırır.
4. İşletim sistemi kontrolü devralır.

3.3 Makinenin Faaliyetlerini Koordine Etme

SINAV · PROGRAM İLE SÜREÇ FARKI (EN TEMEL KAVRAM)

Program: statik bir yönergeler kümesi. **Süreç** *(process)*: bir programın işletim sistemi denetiminde yürütülme faaliyeti. Zamanla özellikleri değişen dinamik bir etkinliktir.
Süreç durumu *(process state)*: Faaliyetin o anki durumu, yani makinenin o andaki "anlık görüntüsü". PC değeri, diğer CPU yazmaçlarının değerleri ve ilgili bellek hücrelerini içerir.

İşletim sistemi, rekabet eden süreçlerin ihtiyaç duyduğu kaynaklara (çevre birimleri, bellek alanı, dosya erişimi, CPU) sahip olmasını, bağımsız süreçlerin birbirine **karışmamasını** ve bilgi alışverişi gereken süreçlerin bunu **yapabilmesini** sağlar.

Süreç yönetimi: Zamanlayıcı ve Dağıtıcı

ZAMANLAYICI · SCHEDULER

Sistemdeki süreçlerin kaydını tutar. Yeni süreçleri havuza **ekler**, tamamlananları **çıkarır**. Bunun için ana bellekte **süreç tablosu** *(process table)* tutar. Her girdide süreç için ayrılan bellek alanı (bellek yöneticisinden alınır), **öncelik** ve sürecin **hazır** ya da **bekliyor** olduğu bilgisi bulunur.

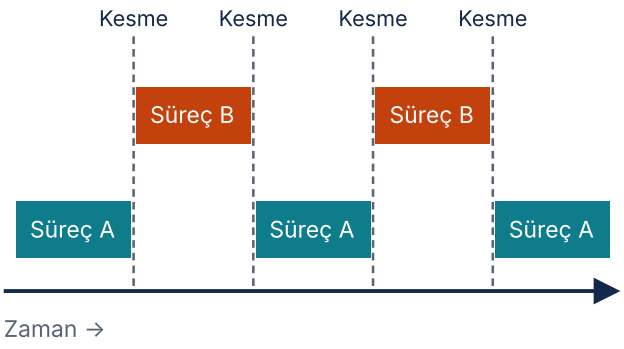
DAĞITICI · DİSPATCHER

Zamanlanmış süreçlerin yürütülmesini gözetir. Zamanı **zaman dilimlerine** *(time slice, ms veya µs)* böler ve CPU'yu süreçler arasında değiştirir. Bir süreçten diğerine geçişe **süreç değişimi** *(process switch / context switch)* denir.

Süreç durumu	Anlamı
Hazır *(ready)*	İlerlemesi devam edebilir durumda.
Bekliyor *(waiting)*	Bir dış olay gerçekleşene dek ilerlemesi durdurulmuş. Örnek: yığın bellek işleminin bitmesi, bir tuşa basılması, başka bir süreçten mesaj gelmesi.

Kesme (Interrupt) mekanizması

1. Dağıtıcı bir sürece zaman dilimi verirken bir **zamanlayıcı devresi** başlatır.
2. Dilim bitince devre bir **kesme** sinyali üretir.
3. CPU **mevcut makine döngüsünü tamamlar**, sürecin durumunu kaydeder ve bellekte önceden belirlenmiş yerdeki **kesme işleyicisini** *(interrupt handler)* çalıştırır. Kesme işleyicisi dağıtıcının bir parçasıdır.
4. Böylece mevcut süreç **kesintiye uğratılır** (preempt) ve kontrol dağıtıcıya geçer.
5. Dağıtıcı, süreç tablosundaki **hazır süreçler içinden en yüksek öncelikliyi** seçer, zamanlayıcıyı yeniden başlatır ve süreci çalıştırır.



Şekil 3.6 — A ve B süreçleri arasında zaman paylaşımı. Her kesme bir süreç değişimidir.

Bir süreç geri döndüğünde ortamı yeniden kurulmalıdır: PC, yazmaçlar ve ilgili bellek hücreleri. Çoklu programlama için tasarlanmış CPU'lar bu durumu kaydetmeyi kesmeye verdikleri tepkinin parçası olarak yapar.

3.4 Süreçler Arası Rekabet

İşletim sistemi kaynakları dağıtır: **dosya yöneticisi** dosya erişimi ve depolama alanı, **bellek yöneticisi** bellek alanı, **zamanlayıcı** süreç tablosunda yer, **dağıtıcı** zaman dilimi tahsis eder.

Örnek (tek yazıcı): Bir süreç yazdırmak isterse işletim sistemi yazıcının başka bir süreç tarafından kullanılıp kullanılmadığına bakar. Boşsa izin verir. Doluysa reddeder ve süreci **bekleyen** olarak sınıflandırır. İki süreç aynı anda yazıcıyı kullansa çıktı ikisi için de işe yaramaz.

Kavram	Tanım

Semafor *Semaphore*	Doğru uygulanmış bir **denetim bayrağı**. Adını demiryolu sinyallerinden alır.
Kritik bölge *Critical Region*	Aynı anda **yalnızca bir süreç** tarafından yürütülmesi gereken komut dizisi. Tek trenin girebileceği ray kesimine benzer.
Karşılıklı dışlama *Mutual Exclusion*	Kritik bölgeye aynı anda yalnızca bir sürecin girmesine izin verilmesi şartı. Kritik bölge bir semaforla korunarak sağlanır.

SEMAFOR NASIL KULLANILIR?

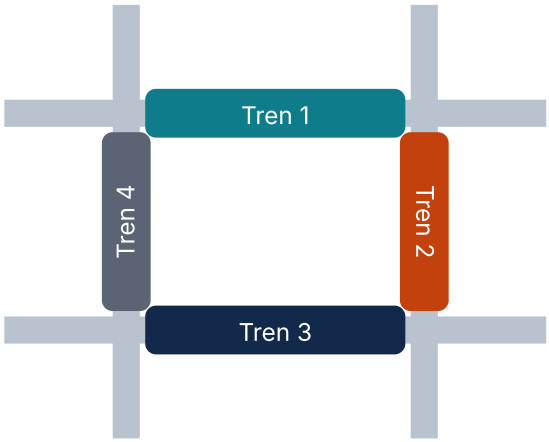
Kritik bölgeye girmek için süreç semaforu **temiz (clear)** bulmalı ve girmeden önce **set** etmelidir. Çıkarken semaforu **temizler**. Semafor **set** durumdaysa giriş yapmak isteyen süreç, semafor temizlenene kadar **bekler**.

Kilitlenme (Deadlock)

İki ya da daha fazla sürecin, her biri **diğerine tahsis edilmiş bir kaynağı beklediği** için ilerleyemediği durumdur. Örnek: Bir süreç yazıcıyı tutup CD sürücüsünü beklerken diğeri CD sürücüsünü tutup yazıcıyı bekler.

SINAV · KİLİTLENMENİN 3 KOŞULU (ÜÇÜ BİRDEN OLMALI)

1. **Paylaşılamayan** kaynaklar için rekabet vardır.
2. Kaynaklar **kısmi olarak** istenir: bir kısmını alan süreç sonra daha fazlasını ister.
3. Bir kaynak tahsis edildikten sonra **zorla geri alınamaz**.



Şekil 3.7 — Paylaşılamayan kavşaklar için rekabet sonucu oluşan kilitlenme. Her tren bir kavşağı tutar ve sıradakini bekler.

Çözüm: Bu 3 koşuldan **herhangi birini** ortadan kaldırmak kilitlenmeyi önler. Diğer yaklaşım, kilitlenmeyi **tespit edip** tahsis edilmiş kaynakları zorla geri alarak düzeltmektir. Örnek: süreç tablosu dolduğu için kilitlenme olursa işletim sistemi bazı süreçleri **sonlandırır** (kill). Böylece tabloda yer açılır, kilitlenme kırılır ve kalan süreçler devam eder.

3.5 Güvenlik

DIŞARIDAN SALDIRILAR

- **Güvensiz parolalar**
- **Dinleme yazılımı** *(sniffing software)*: Faaliyetleri kaydedip sonra saldırgana raporlar. Örneğin işletim sisteminin oturum açma ekranını taklit eden bir program.
- **Önlem · Denetim yazılımı** *(auditing software)*: Sistemdeki faaliyetleri kaydedip analiz eder. Yanlış parolayla giriş denemelerini ve kullanıcının geçmiş davranışına uymayan hesap faaliyetlerini tespit eder.

İÇERİDEN SALDIRILAR

Saldırgan sisteme girdikten sonra ilgi çekici bilgi ya da zararlı yazılım yerleştirecek yer arar. Buna karşı CPU'lar iki **ayrıcalık düzeyinde** çalışır:

- **Ayrıcalıklı mod** *(privileged)*: Makine dilindeki **tüm** komutlar çalıştırılabilir.
- **Ayrıcalıksız mod** *(nonprivileged)*: Kabul edilen komut listesi **sınırlıdır**.

- **Ayrıcalıklı komutlar** *(privileged instructions)*: Yalnızca ayrıcalıklı modda çalışabilen komutlar. Örnek: **bellek sınır yazmaçlarının** içeriğini değiştiren ve CPU'nun **ayrıcalık modunu değiştiren** komutlar.
- Ayrıcalıksız modda ayrıcalıklı bir komut çalıştırılmaya çalışılırsa **kesme** oluşur. Kesme CPU'yu ayrıcalıklı moda alır ve kontrolü işletim sistemindeki kesme işleyicisine verir.
- CPU **ilk açıldığında ayrıcalıklı moddadır**. İşletim sistemi önyükleme sonunda tüm komutları çalıştırabilir. Bir sürece zaman dilimi verdiği her seferde “ayrıcalık modunu değiştir” komutuyla CPU'yu **ayrıcalıksız moda** geçirir.

Bölüm 3 — Çözümlü Sorular

1. Kuyruk örnekleri verin. Her birinde FIFO yapısını bozan durumu belirtin.

Cevap: Klasik örnek: bir etkinliğe bilet almak için bekleyen insan sırası. Biri “araya kaynarsa” FIFO yapısı bozulur.

2. Hangileri gerçek zamanlı işleme gerektirir? a) Posta etiketi yazdırma b) Bilgisayar oyunu oynama c) Akıllı telefonda tuşlanan numaraları ekranda gösterme d) Gelecek yılın ekonomisini tahmin eden program e) MP3 kaydı çalma

Cevap: **(b), (c) ve (e)**.

3. Zaman paylaşımı ile çok görevlilik arasındaki fark nedir?

Cevap: Zaman paylaşımı, birden fazla kullanıcının bir makineye aynı anda erişmesidir. Çok görevlilik, bir kullanıcının aynı anda birden fazla görev yürütmesidir.

4. Tipik bir işletim sisteminin bileşenlerini listeleyip her birinin rolünü tek cümleyle özetleyin.

Cevap: **Kabuk:** makinenin çevresiyle iletişim kurar. **Dosya yöneticisi:** yığın belleğin kullanımını koordine eder. **Aygıt sürücüleri:** çevre birimleriyle iletişimi yürütür. **Bellek yöneticisi:** ana belleğin kullanımını koordine eder. **Zamanlayıcı:** sistemdeki süreçleri koordine eder. **Dağıtıcı:** süreçlere CPU zamanı atanmasını denetler.

5. Uygulama yazılımı ile yardımcı yazılım arasındaki fark nedir?

Cevap: Çizgi belirsizdir. Kabaca yardımcı yazılım temel ve evrensel görevleri, uygulama yazılımı ise makinenin kullanım amacına özgü görevleri yapar.

6. Sanal bellek nedir?

Cevap: Veri ve programların ana bellek ile yığın bellek arasında gidip gelmesiyle (sayfalama) varmış gibi görünen hayali bellek alanıdır.

7. Önyükleme sürecini özetleyin.

Cevap: Makine açılınca CPU, ROM'daki önyükleyiciyi çalıştırır. Önyükleyici, işletim sistemini yığın bellekten ana belleğin uçucu kısmına aktarır. Aktarım bitince CPU'yu işletim sistemine atlatır (JUMP).

8. Program ile süreç arasındaki farkı özetleyin.

Cevap: Program bir yönergeler kümesidir. Süreç ise bu yönergeleri izleme eylemidir.

9. Bir kesme oluştuğunda CPU hangi adımları uygular?

Cevap: Mevcut makine döngüsünü tamamlar, mevcut sürecin durumunu kaydeder ve PC'yi önceden belirlenmiş bir değere (kesme işleyicisinin adresine) ayarlar. Böylece sıradaki komut kesme işleyicisinin ilk komutu olur.

10. Çoklu programlamalı sistemde yüksek öncelikli süreçler nasıl daha hızlı çalıştırılabilir?

Cevap: Daha yüksek öncelik verilerek dağıtıcının onları tercih etmesi sağlanabilir. Diğer seçenek, yüksek öncelikli süreçlere daha uzun zaman dilimleri vermektir.

11. Zaman dilimi 50 ms ve her süreç değişimi en fazla 1 µs ise makine bir saniyede kaç sürece hizmet verebilir?

Cevap: Her süreç dilimini tamamen kullanırsa saniyede **yaklaşık 20 sürece** (1 s / 50 ms = 20) tam dilim verilebilir. Süreçler dilimlerini tamamen kullanmazsa bu sayı çok daha yüksek olabilir. Ancak o zaman süreç değişimi için harcanan süre önem kazanır.

12. (Ek) Kilitlenmenin oluşması için hangi üç koşul sağlanmalıdır? Nasıl giderilir?

Cevap: Paylaşılamayan kaynak rekabeti, kaynakların kısmi istenmesi ve tahsis edilen kaynağın zorla geri alınamaması. Bu koşullardan biri ortadan kaldırılarak önlenir ya da tespit edilip kaynaklar zorla geri alınarak (ör. süreç sonlandırma) giderilir.

13. (Ek) Ayrıcalıksız moddaki bir süreç ayrıcalıklı bir komut çalıştırmaya çalışırsa ne olur?

Cevap: Bir kesme oluşur. CPU ayrıcalıklı moda geçer ve kontrol işletim sisteminin kesme işleyicisine aktarılır.

★ Son Tekrar — Sık Karıştırılan Kavramlar

Kavram çifti	Fark
ALU ↔ Denetim birimi	ALU **işlemi yapar**. Denetim birimi faaliyetleri **koordine eder**.
Genel ↔ Özel amaçlı yazmaç	Genel: işlenen veri. Özel: PC, IR gibi belirli görevler.
PC ↔ IR	PC = **sıradaki** komutun **adresi**. IR = **şu an** yürütülen **komutun kendisi**.
LOAD ↔ STORE	LOAD: bellek → yazmaç. STORE: yazmaç → bellek.
1RXY ↔ 2RXY	1: XY **adresindeki** değer. 2: XY **değerinin kendisi**.
RISC ↔ CISC	RISC: az, basit, hızlı komut (ARM). CISC: çok, güçlü komut (Intel).
Döndürme ↔ Mantıksal ↔ Aritmetik kaydırma	Döndürme: düşen bit öbür uca. Mantıksal: boşluğa 0. Aritmetik: işaret biti korunur.
AND ↔ OR ↔ XOR maskesi	AND: bit temizle (0'la). OR: bit kur (1'le). XOR: bit tersle.
Paralel ↔ Seri iletişim	Paralel: çok yol, aynı anda. Seri: tek yol, art arda.
SISD ↔ SIMD ↔ MIMD	SISD: paralellik yok. SIMD: aynı program, farklı veri. MIMD: farklı program, farklı veri.
Boru hattı ↔ Paralel işleme	Boru hattı: tek işlemcide döngü adımlarını üst üste bindirme. Paralel: birden çok işlemci.
Toplu ↔ Etkileşimli işleme	Toplu: kuyrukta bekle, etkileşim yok. Etkileşimli: kullanıcıyla diyalog (gerçek zamanlı).
Zaman paylaşımı ↔ Çok görevlilik	Çok kullanıcı ↔ tek kullanıcının çok görevi.
Kullanıcı arayüzü ↔ Çekirdek	Arayüz: kullanıcıyla iletişim (shell/GUI). Çekirdek: temel iç işlevler.
Zamanlayıcı ↔ Dağıtıcı	Zamanlayıcı: süreç tablosuna ekle/çıkar. Dağıtıcı: zaman dilimi tahsis et.
Program ↔ Süreç	Statik yönerge ↔ yürütülmekte olan dinamik faaliyet.
Hazır ↔ Bekleyen süreç	Hazır: devam edebilir. Bekleyen: dış bir olayı bekliyor.
ROM ↔ Ana bellek (RAM)	ROM: uçucu değil, salt okunur (önyükleyici burada). Ana bellek: uçucu.
Ayrıcalıklı ↔ Ayrıcalıksız mod	Tüm komutlar ↔ sınırlı komut listesi.

A–Z Terim Sözlüğü (İngilizce → Türkçe)

İngilizce	Türkçe
Arithmetic/Logic Unit	Aritmetik/Mantık Birimi
Bandwidth	Bant genişliği
Interrupt / Interrupt handler	Kesme / Kesme işleyicisi

Batch processing	Toplu işleme
Benchmark	Kıyaslama programı
Boot loader	Önyükleyici
Bus	Veri yolu
Cache memory	Önbellek
Circular shift / Rotation	Dairesel kaydırma / Döndürme
Control unit	Denetim birimi
Controller	Denetleyici
Critical region	Kritik bölge
Deadlock	Kilitlenme
Device driver	Aygıt sürücüsü
Direct Memory Access (DMA)	Doğrudan bellek erişimi
Dispatcher	Dağıtıcı
Fetch / Decode / Execute	Getir / Çöz / Yürüt
File manager	Dosya yöneticisi
Firmware	Donanım yazılımı
Handshaking	El sıkışma
Instruction register	Komut yazmacı

Kernel	Çekirdek
Load balancing / Scaling	Yük dengeleme / Ölçekleme
Machine cycle	Makine döngüsü
Mask / Masking	Maske / Maskeleme
Memory manager	Bellek yöneticisi
Multiprogramming	Çoklu programlama
Mutual exclusion	Karşılıklı dışlama
Op-code / Operand	İşlem kodu / İşlenen
Paging / Page	Sayfalama / Sayfa
Pipelining	Boru hattı
Privileged instruction	Ayrıcalıklı komut
Process / Process table	Süreç / Süreç tablosu
Process (context) switch	Süreç değişimi
Program counter	Program sayacı
Real-time processing	Gerçek zamanlı işleme
Register	Yazmaç
Scheduler	Zamanlayıcı
Semaphore	Semafor
Time slice / Virtual memory	Zaman dilimi / Sanal bellek

SINAVDAN ÖNCEKİ SON 10 DAKİKA

1. CPU = ALU + Denetim birimi + Yazmaçlar
2. Kayıtlı program kavramı
3. Komut türleri: veri aktarımı, aritmetik/mantık, denetim
4. Ek C op-code tablosu (1–C) + 35A7, B258 çözümü
5. Makine döngüsü: getir (PC += 2), çöz, yürüt
6. Maske kuralları: AND 0'lar, OR 1'ler, XOR tersler
7. Döndürme: sağa n = sola 8 − n
8. DMA, el sıkışma, seri/paralel, SIMD/MIMD
9. Çekirdek bileşenleri + sanal bellek + önyükleme
10. Program/süreç, kesme adımları, semafor, kilitlenmenin 3 koşulu